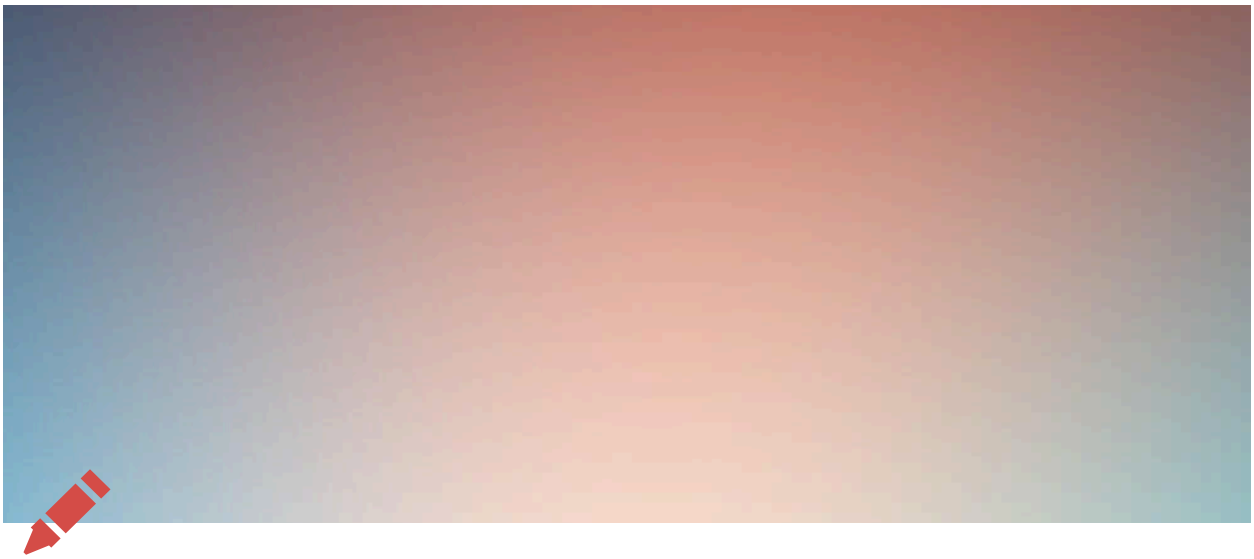


ER SCHEMA TERMS EXPLANATION

ENTITY AND ENTITY TYPE

Entity is any thing in system that stores independently and i want to store data about . Everything is entity

- A user named "Rahul Sharma" → that's an entity
- IndiGo Airlines → that's an entity
- The Taj Mahal Palace Hotel → that's an entity
- A booking made on 15th March → that's an entity

Entity type is the category .

For example , USER is the entity type , then “rahul sharma” is one specific entity or an instance of that entity type

In ER diagram entity types are shown as **rectangles.**

STRONG ENTITY AND WEAK ENTITY

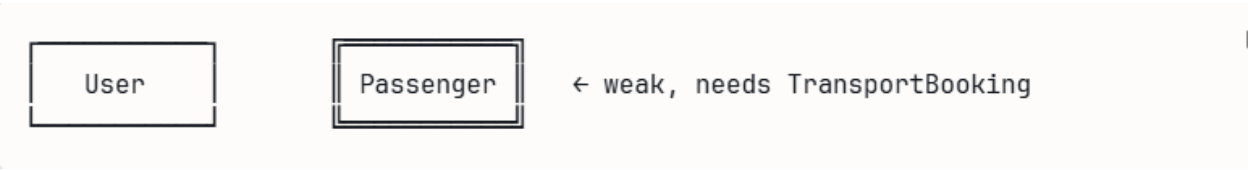
Strong Entity : EXISTS ON ITS OWN .

For example , `User` entity exists whether they have a booking or not , `Hotel` entity exists whether someone booked it or not they exist independently

Weak Entity : CANNOT EXIST WITHOUT A PARENT ENTITY

For example , `passenger` entity cannot exist without `Transportbooking` entity as passenger only exists if that passenger booked some kind of transport . There can not be an entity `RoomAvailability` without `Roomcategory` as we select room category first then room availability is shown .

In ER diagrams, weak entities are drawn as **double rectangles**.



ATTRIBUTES

Attributes are properties of an entity

Simple Attribute

Cannot be broken down further. It's atomic. `age` , `gender` , `departure_date` , `star_rating`

Composite Attribute

Can be broken into smaller meaningful parts. `full_address` in `Hotel` table — it could be broken into `street` , `city` , `state` , `pincode`

Key Attribute

The attribute that **uniquely identifies** every entity of that type. No two users can have the same `user_id`. No two hotels can have the same `hotel_id`.[PRIMARY KEY]

Multi-valued Attribute

An attribute that can hold **multiple values** for a single entity.
Think about `amenities` in our `Hotel` table. One hotel can have: "Swimming Pool, Gym, Spa, Restaurant, Parking" — that's multiple values for one attribute.

In ER diagrams, multi-valued attributes are drawn with **double ovals**.

"Multi-valued attributes → Separate Relation and FK".

When we convert this ER TO RELATIONAL SQL , multi valued attributes becomes its own individual table .
So `amenities` could become a separate `HotelAmenity` table with a FK back to `Hotel` and attributes with “Gym” , “Spa” , etc. in that table

Derived Attribute

A value that can be from other data. We don't need to store it. For example ,

`duration_of_stay` in a hotel booking — you can calculate it as `checkout_date - checkin_date`.

`age` of a passenger , technically derivable from date of birth but we store it for convenience annd for better structured queiries.

> In ER diagrams, derived attributes are drawn with **dashed ovals**.

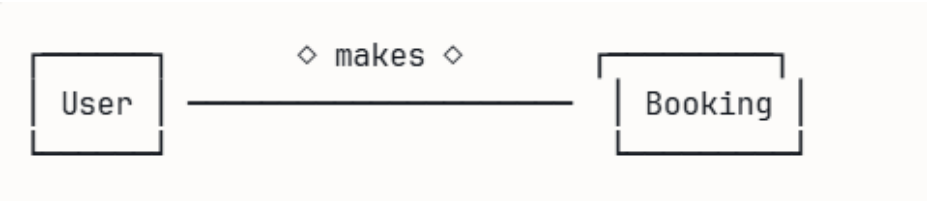
RELATIONSHIP AND RELATIONSHIP TYPE

Relationship is an association between two or more entities.

- A `User` **makes** a `Booking` — that's a relationship
- A `Vendor` **lists** a `Hotel` — that's a relationship
- A `Booking` **covers** a `ScheduleDeparture` — that's a relationship

A **Relationship Type** is the category — just like entity type. "makes" is a relationship type between `User` and `Booking`.

In ER diagrams, relationship types are drawn as **diamonds**



CARDINALITY AND TYPES

Cardinality tells you **how many** of one entity can relate to **how many** of another.

1:1 — One to One

One entity on the left relates to exactly one entity on the right, and vice versa.

For example `Booking` and `TransportBooking` one booking has exactly one transport booking detail and one transport booking detail belongs to exactly one booking .

`Booking` and `CancellationRefund` , for every cancellation refund request there is single booking and for every booking there can at most 1 cancellation refund request so thats 1:1

1:N — One to Many

One entity one the left relates to MANY on right.

One `Vendor` lists many `Hotel`s. But each `Hotel` belongs to one `Vendor`. → 1:N
One `Hotel` has many `RoomCategory`s. Each `RoomCategory` belongs to one `Hotel`. → 1:N
One `RouteSchedule` generates many `ScheduleDeparture`s. → 1:N

M:N — Many to Many

Many on the left relate to many on the right.

One `User` can write many `Review`s. One `Hotel` can receive many `Review`s. A user can review many hotels. → M:N between User and Hotel (through Review) . Remember 1:N between users and reviews as one user ⟶ multiple reviews but one review ⟶ one user . But multiple users can review multiple hotels so M:N

"M:N relationship type → Relationship maps to Relation and two FKs". When you convert M:N to SQL, the relationship itself becomes a **junction table** with two foreign keys.

IN 1:N OR N:1
we just write `user` reviews `hotel` through writes in diamond or `vender` lists `hotel` with lists in diamond .

IN M:N
there is separate schema table in relational schema
For example , `user` and `hotels` through `reviews` , here reviews is a table with two foreign keys `user_id` referencing user entity and `hotel_id` referencing hotel entity

Participation Constraints

Total Participation

Every entity must participate in this relationship. Drawn as a double line between entities . For example ,

Every `Passenger` MUST belong to a `TransportBooking`. A passenger with no booking cannot exist. That's total participation.
Every `RoomAvailability` MUST belong to a `RoomCategory`. Total participation.

Partial Participation

An entity MAY or MAY NOT participate. Drawn as a single line between entities . For example,

A `User` may or may not have made a `Booking` yet. “makes" in diamond as relationship entity . A brand new user with zero bookings still exists in the system. That's partial participation of User in the "makes" relationship.

A `Hotel` may or may not have received a `Review` yet. Partial participation.

N-ary Relationships

Most relationships are **binary** — between exactly 2 entity types. But sometimes 3 or more entities are involved in one relationship simultaneously

We may not have a ternary (3-way relationship) but to understand we take this example

A `User` books a `ScheduleDeparture` in a specific `ScheduleClass` — three entities involved simultaneously in one booking event. Here booking table will be created in SQL or relational schema where three foreign keys for `user_id , schedule_departure_id, class_id`
in the booking table.
We have broken it down to 2+1 as `user_id and  TransportBooking` in which transport booking has `departure_id` and implicitly the class is already baked into the departure.

Aggregation

Aggregation means treating a relationship as it were an entity so that another relationship can connect to it

A `User` **books** a (`ScheduleDeparture` + `ScheduleClass`) combination — and then **Insurance** is taken out ON that booking which is actually a relationship not an entity

`Booking` is the aggregated entity that Insurance, CancellationRefund, and Review all connect to.

ER SCHEMA VS RELATIONAL SCHEMA

Explanation of concepts with respect to travel_booking_system database

ER Concept	What happens in SQL	Your System Example
Entity type	Becomes a table	`User`, `Hotel`, `Vendor`
Simple Attribute	Becomes a column	`full_name`, `email`, `star_rating`
Key attribute	Becomes PRIMARY KEY	`user_id`, `hotel_id`
Composite attribute	Split into multiple columns	`full_address` → `street`, `city`, `pincode`
Multi-valued attribute	Becomes its own table with FK	`amenities` → `HotelAmenity(hotel_id, amenity)`
1:1 relationship	FK on the total participation side	`CancellationRefund.booking_id` → `Booking`
1:N relationship	FK on the N-side (many side)	`Hotel.vendor_id` → `Vendor` (N hotels, 1 vendor)
M:N relationship	New junction table with 2 FKs	`Review(user_id, entity_id)`
N-ary relationship	New table with N foreign keys	`TransportBooking(booking_id, departure_id, class_id)`

ER Concept	What happens in SQL	Your System Example
Aggregation	Treat the relationship box as entity	`Booking` becomes the entity that Insurance connects to

Note : we may have not included booking but used it to understand these concepts

```
REAL WORLD                    ER DIAGRAM              SQL TABLE
──────────────                ──────────              ─────────
"A thing that exists"    →    Rectangle          →    Table
"A property of a thing"  →    Oval               →    Column
"Uniquely identifies it" →    Underlined oval    →    PRIMARY KEY
"Links to another table" →    Diamond            →    FOREIGN KEY
"Must exist together"    →    Double line        →    NOT NULL + FK
"One owns many"          →    1:N with arrow     →    FK on the many side
"Many relate to many"    →    M:N diamond        →    Junction table
"Can't exist alone"      →    Double rectangle   →    Weak entity + FK
```